

Modul 04: Services & Signals

<i>Ziel:</i>	<i>Services verwenden</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~60min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>05.08.2024</i>

1 Service erstellen und verwenden

Komponenten sollten Daten nicht direkt abrufen oder speichern und schon gar nicht Fake-Daten präsentieren. Sie sollten sich ausschließlich auf die Darstellung der Daten konzentrieren. Der Datenzugriff wird mit Diensten ermöglicht, weshalb wir jetzt einen Task-Service erstellen.

```
ng generate service services/task
```

Im wesentlichen verschieben wir die Logik von der task-list Komponente in den Service. Später können wir den Service um die weiteren CRUD Methoden update und delete ergänzen.

```
@Injectable({
  providedIn: 'root',
})
export class TaskService {
  items: TaskItem[] = [
    { title: 'Angular Kurs absolvieren' } as TaskItem
  ];

  addTask(task: Omit<TaskItem, 'id' | 'completed'>) {
    this.items.push({
      ...task,
      id: `${Date.now()}`,
      completed: false,
    });
  }
}
```

Um den Service zu verwenden, nutzen wir Dependency Injection um ihn in der Komponente nutzen zu können. Wir müssen lediglich den Service als Argument notieren und Angular wird bei Instanzieren der Klasse diese Abhängigkeit automatisch auflösen.

```
constructor(public taskService: TaskService) {}
```

Wir aktualisieren das HTML-Template entsprechend.

```
@for (task of taskService.items; track $index) {
  <li title="#{{ $index }}">
    <app-task-item [item]="task" />
  </li>
}
```

2 Form Komponente erstellen

Jetzt können wir das Formular in eine eigene Komponente extrahieren, welche ebenfalls auf den Service zugreift. Die task-list Komponente kümmert sich ausschließlich um das Anzeigen der Liste während sich die Form Komponente um das Erstellen von Aufgaben kümmern wird.

```
constructor(private service: TaskService) {}

addTask(e: Event, title: string) {
  e.preventDefault();

  if (!this.valid) {
    this.setStatus('Bitte geben Sie den Titel der Aufgabe ein.');
```

```
  } else {
    this.service.addTask({
      title,
      priority: 'default',
    });

    const list = this.service.items();
    this.setStatus(`${list.length} Aufgaben insgesamt`);

    this.formRef.nativeElement.reset();
  }
}
```

Die Komponente enthält noch das Status-Element, was unschön ist. Das können wir in einem späteren Schritt beheben, indem wir ein Toast-Message-System einführen werden.

Nun stellen wir plötzlich fest, dass die Aufgabenliste sich nicht mehr aktualisiert, wenn wir eine neue Aufgabe hinzufügen. Um das zu beheben, müssen wir irgendwie das Datenarray auf Änderungen überwachen, indem wir geschickt Hooks des Page Lifecycles verwenden.

Deutlich eleganter lässt sich das Problem mit einem BehaviourSubject und Observables aus dem Paket RxJS lösen. Wir werden im nächsten Schritt jedoch keine dieser Lösungen umsetzen, da es seit Angular 17 einen deutlich einfacheren Weg gibt.

3 Reaktivität mit Signals

Seit Angular 17 gibt es eine sehr einfache Möglichkeit die Reaktivität unserer App zu gewährleisten. Die Handhabung ist deutlich einfacher als mit Observables oder dem Page Lifecycle zu hantieren.

```
@Injectable({
  providedIn: 'root',
})
export class TaskService {
  readonly items = signal<TaskItem[]>([
    { title: 'Angular Kurs absolvieren' } as TaskItem
  ]);

  addTask(task: Omit<TaskItem, 'id' | 'completed'>) {
    this.items.update((tasks) => [
      ...tasks,
      {
        ...task,
        id: `${Date.now()}`,
        completed: false,
      },
    ]);
  }
}
```

Wir aktualisieren das HTML Template entsprechend. Das Signal wird aufgelöst, indem es wie eine function aufgerufen wird.

```
@for (task of taskService.items(); track $index) {
  <li title="#{{ $index }}">
    <app-task-item [item]="task" />
  </li>
}
```

4 Toast-Message-System erstellen

Es soll eine shared Toast-Komponente erstellt werden. Sie enthält das Status-Element und kann mehrere Einheiten untereinander darstellen. Die Nachrichten sollen nach drei Sekunden automatisch verschwinden. Das Stylesheet dafür sieht so aus:

```
.list {
  list-style: none;
  position: absolute;
  right: 0;
```

```
    top: 0;
  }

  .list > .toast {
    margin-bottom: .4rem;
  }
```

Die Nachrichten können über einen Toast-Service gesendet werden. In der `app.component.html` soll die Toast-Komponente eingebunden werden und vollkommen autark sich um die Nachrichten kümmern. Eine Toast-Message soll so aussehen:

```
export interface ToastMessage {
  type: 'error' | 'info' | 'warning';
  title: string;
  message: string;
}
```

Im Service sollen dazu analog die Methoden `sendInfo()` und `sendError()` existieren, welche nur die `message` als Parameter annehmen und einreihen.